

Guía Práctica: Integridad Referencial en SQL

Comportamiento de ON DELETE y ON UPDATE

March 24, 2026

1 Introducción

La integridad referencial garantiza que las relaciones entre las tablas de una base de datos se mantengan consistentes. Las cláusulas `ON DELETE` y `ON UPDATE` definen el comportamiento de los registros dependientes (claves foráneas) cuando el registro principal es modificado o eliminado.

Esta guía presenta un entorno de pruebas estructurado en cuatro niveles jerárquicos: Departamentos, Profesores, Módulos y Matrículas.

2 Creación del Esquema e Inserción de Datos

Ejecuta el siguiente bloque (compatible con MariaDB/MySQL y PostgreSQL) para preparar el entorno de pruebas.

```
DROP DATABASE IF EXISTS integridad_referencial;
CREATE DATABASE integridad_referencial;
USE integridad_referencial;

-- 1. Tabla Padre (Nivel 1)
CREATE TABLE departamentos (
    id_dept INT PRIMARY KEY,
    nombre VARCHAR(50)
);

-- 2. Tabla Intermedia (Nivel 2)
CREATE TABLE profesores (
    id_prof INT PRIMARY KEY,
    nombre VARCHAR(50),
    id_dept INT,
    CONSTRAINT fk_prof_dept FOREIGN KEY (id_dept)
        REFERENCES departamentos(id_dept)
        ON DELETE SET NULL
);

-- 3. Tabla Intermedia (Nivel 3)
CREATE TABLE modulos (
    id_mod INT PRIMARY KEY,
```

```

siglas VARCHAR(10),
id_prof INT,
CONSTRAINT fk_mod_prof FOREIGN KEY (id_prof)
    REFERENCES profesores(id_prof)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

-- 4. Tabla Hija Final (Nivel 4)
CREATE TABLE matriculas (
    id_matricula INT PRIMARY KEY,
    alumno VARCHAR(50),
    id_mod INT,
    CONSTRAINT fk_mat_mod FOREIGN KEY (id_mod)
        REFERENCES modulos(id_mod)
        ON DELETE CASCADE
);

-- INSERCIÓN DE DATOS DE PRUEBA
INSERT INTO departamentos VALUES (10, 'Informatica y Comunicaciones'), (20, '
    Formacion y Orientacion Laboral');

INSERT INTO profesores VALUES (1, 'Ada', 10), (2, 'Alan', 10), (3, 'Grace', 20);

INSERT INTO modulos VALUES (101, 'GBD', 1), (102, 'ASO', 1), (103, 'LMSGI', 2);

INSERT INTO matriculas VALUES (1001, 'Estudiante A', 101), (1002, 'Estudiante B',
    101), (1003, 'Estudiante C', 103);

```

3 Laboratorio de Operaciones

3.1 Caso A: ON UPDATE CASCADE (Actualización en cascada)

Modificamos la clave primaria de un registro padre. La base de datos propaga el cambio automáticamente a las tablas hijas para no romper la relación.

```

SELECT * FROM modulos;
-- Actualizamos el ID del profesor Ada
UPDATE profesores SET id_prof = 99 WHERE id_prof = 1;
-- Los modulos 101 y 102 ahora tienen fk_prof = 99
SELECT * FROM modulos;

```

3.2 Caso B: ON DELETE RESTRICT (Bloqueo de seguridad)

Evita la eliminación de un registro si este tiene dependencias activas.

```

-- Intentamos eliminar a Alan, que imparte LMSGI
DELETE FROM profesores WHERE id_prof = 2;
-- RESULTADO: ERROR. El sistema bloquea la operacion.

```

3.3 Caso C: ON DELETE SET NULL (Puesta a nulo)

Si el registro padre desaparece, la clave foránea en los registros hijos adquiere el valor NULL, manteniéndose el registro hijo.

```
SELECT * FROM profesores;
-- Eliminamos el departamento de FOL
DELETE FROM departamentos WHERE id_dept = 20;
-- Grace (id_prof 3) sigue existiendo, pero su id_dept es NULL
SELECT * FROM profesores;
```

3.4 Caso D: ON DELETE CASCADE (Borrado en cadena)

La eliminación del registro padre desencadena la destrucción automática de todos los registros hijos asociados.

```
SELECT * FROM matriculas;
-- Retiramos el modulo GBD del catalogo
DELETE FROM modulos WHERE id_mod = 101;
-- Las matriculas 1001 y 1002 han sido eliminadas fisicamente
SELECT * FROM matriculas;
```

4 Criterios de Elección: ¿Cuándo utilizar cada opción?

La elección del comportamiento de las claves foráneas en un diseño físico depende completamente de la lógica de negocio y de la naturaleza de la relación entre las entidades (diagrama Entidad-Relación). A continuación se detallan las buenas prácticas para cada caso:

4.1 RESTRICT / NO ACTION (Opción por defecto y de seguridad)

- **¿Cuándo usarlo?:** Cuando los registros dependientes contienen información histórica, legal o vital que no debe desaparecer ni quedar "huérfana" de forma accidental. Obliga al administrador de la base de datos a tomar una decisión consciente (reasignar los hijos o borrarlos a mano) antes de eliminar el padre.
- **Ejemplo clásico:** *Clientes y Facturas*. Un cliente no debe poder borrarse jamás si tiene facturas emitidas a su nombre. El sistema debe bloquear la operación.

4.2 CASCADE (Dependencia Existencial estricta)

- **¿Cuándo usarlo?:** Cuando la entidad hija es una *entidad débil* por identificación; es decir, su existencia no tiene ningún sentido lógico si desaparece la entidad padre a la que pertenece.
- **Ejemplo clásico:** *Facturas y Líneas de Factura*. Si se elimina una factura del sistema, todas las líneas de detalle que la componen deben desaparecer fulminantemente.

4.3 SET NULL (Relaciones Opcionales o Temporales)

- **¿Cuándo usarlo?:** Cuando la entidad hija tiene entidad propia y puede existir de forma totalmente independiente. La relación con la tabla padre representa una asignación o estado temporal. Requisito imprescindible: la columna de la clave foránea debe admitir valores NULL.
- **Ejemplo clásico:** *Empleados y Equipos Informáticos*. Si un empleado es dado de baja de la empresa, su registro se elimina, pero el ordenador portátil que tenía asignado sigue existiendo físicamente en el inventario. Su campo `id_empleado` pasará a ser NULL (equipo disponible).